

Naive Bayes Classifiers

1. What is Naive Bayes?

Naive Bayes is a classification algorithm based on Bayes' Theorem. Its main assumption is that all features are independent — that's why it's called "Naive."

The basic formula (Bayes' Theorem) is:

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

Where:

- $P(C|X)$ → Probability that a sample belongs to class C given features X
- $P(X|C)$ → Probability of observing features X given class C
- $P(C)$ → Prior probability of class C
- $P(X)$ → Probability of features X (usually ignored in classification)

2. Example: Predicting a class label using naïve Bayesian classification. We wish to predict the class label of a tuple using naïve Bayesian classification

$X = (\text{age} = \text{youth}, \text{income} = \text{medium}, \text{student} = \text{yes}, \text{credit_rating} = \text{fair}).$

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

1. The prior probability of each class, can be computed based on the training tuples:

$$P(C) \begin{cases} P(\text{buys_computer} = \text{yes}) = 9/14 = 0.643 \\ P(\text{buys_computer} = \text{no}) = 5/14 = 0.357. \end{cases}$$

2. Compute $P(X|C_i)$, for $i = 1, 2$, we compute the following conditional probabilities:

حسب
وتكرر العملية
لكل الخصائص

$$\begin{aligned} P(\text{age} = \text{youth} \mid \text{buys_computer} = \text{yes}) &= 2/9 = 0.222 \\ P(\text{age} = \text{youth} \mid \text{buys_computer} = \text{no}) &= 3/5 = 0.600 \\ P(\text{income} = \text{medium} \mid \text{buys_computer} = \text{yes}) &= 4/9 = 0.444 \\ P(\text{income} = \text{medium} \mid \text{buys_computer} = \text{no}) &= 2/5 = 0.400 \\ P(\text{student} = \text{yes} \mid \text{buys_computer} = \text{yes}) &= 6/9 = 0.667 \\ P(\text{student} = \text{yes} \mid \text{buys_computer} = \text{no}) &= 1/5 = 0.200 \\ P(\text{credit_rating} = \text{fair} \mid \text{buys_computer} = \text{yes}) &= 6/9 = 0.667 \\ P(\text{credit_rating} = \text{fair} \mid \text{buys_computer} = \text{no}) &= 2/5 = 0.400. \end{aligned}$$

Using these probabilities, we obtain

$P(X|C_{\text{yes}})$

$$\begin{aligned} P(X \mid \text{buys_computer} = \text{yes}) &= P(\text{age} = \text{youth} \mid \text{buys_computer} = \text{yes}) \\ &\times P(\text{income} = \text{medium} \mid \text{buys_computer} = \text{yes}) \\ &\times P(\text{student} = \text{yes} \mid \text{buys_computer} = \text{yes}) \\ &\times P(\text{credit_rating} = \text{fair} \mid \text{buys_computer} = \text{yes}) \\ &= 0.222 \times 0.444 \times 0.667 \times 0.667 = \boxed{0.044}. \end{aligned}$$

نعم
 $P|X_{\text{yes}}$

Similarly,

$$P(X \mid \text{buys_computer} = \text{no}) = 0.600 \times 0.400 \times 0.200 \times 0.400 = \boxed{0.019}.$$

نعم
 $P|X_{\text{no}}$

3. To find the class, C_i , that maximizes $P(X|C) P(C)$, we compute

$$P(X \mid \text{buys_computer} = \text{yes}) P(\text{buys_computer} = \text{yes}) = 0.044 \times 0.643 = 0.028$$

$$P(X \mid \text{buys_computer} = \text{no}) P(\text{buys_computer} = \text{no}) = 0.019 \times 0.357 = 0.007.$$

$P(X|C) P(C)$ Yes

$P(X|C) P(C)$ No

Therefore the naïve Bayesian classifier predicts $\text{buys_computer} = \text{yes}$ for tuple X .

3. Why use Naive Bayes?

المحاسب/الوقت

- Very fast to train even on large datasets.
- Performs surprisingly well for text classification.
- Simple to implement.

سريع التدريب
النتائج سوف تتحسن (دول)

تصنيف النصوص

سهولة التنفيذ

Limitations:

المساوي

- The independence assumption is rarely true in real life

لا يفترض استقلال الخصائص
نادراً ما تكون صحيحة للواقع

4. Types of Naive Bayes in scikit-learn

أنواع

1. GaussianNB

كاسية

- Works with continuous data.
- Assumes features follow a Gaussian (normal) distribution.
- Example: Predicting if a flower is a species based on petal lengths and widths.

البيانات المستمرة

النوع زيج العادي

2. BernoulliNB

بيرنولي

- Works with binary features (0 or 1). *نصفية المصروفات*
- Commonly used in text classification, e.g., whether a word appears in a document. *10/27*

3. MultinomialNB

- Works with count data (non-negative integers). *حساب البيانات*
- Often used in text classification for word frequencies. *البيانات الكمية*
- Example: Counting how many times each word appears in emails for spam detection. *تصنيف الرسائل الإلكترونية*

برنامج باستخدام GaussianNB

```

from sklearn.naive_bayes import GaussianNB
# بيانات التدريب بعد الترميز
# age: youth=0, middle_aged=1, senior=2
# income: low=0, medium=1, high=2
# student: no=0, yes=1
# credit_rating: fair=0, excellent=1
# buys_computer (y): no=0, yes=1
X = [
    [0, 2, 0, 0], # 1: youth, high, no, fair
    [0, 2, 0, 1], # 2: youth, high, no, excellent
    [1, 2, 0, 0], # 3: middle_aged, high, no, fair
    [2, 1, 0, 0], # 4: senior, medium, no, fair
    [2, 0, 1, 0], # 5: senior, low, yes, fair
    [2, 0, 1, 1], # 6: senior, low, yes, excellent
    [1, 0, 1, 1], # 7: middle_aged, low, yes, excellent
    [0, 1, 0, 0], # 8: youth, medium, no, fair
    [0, 0, 1, 0], # 9: youth, low, yes, fair
    [2, 1, 1, 0], # 10: senior, medium, yes, fair
    [0, 1, 1, 1], # 11: youth, medium, yes, excellent
    [1, 1, 0, 1], # 12: middle_aged, medium, no, excellent
    [1, 2, 1, 0], # 13: middle_aged, high, yes, fair
    [2, 1, 0, 1], # 14: senior, medium, no, excellent
]
y = [0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 0] # buys_computer: no=0, yes=1

# إنشاء وتدريب نموذج Naive Bayes
m = GaussianNB()
m.fit(X, y)

# المثال الجديد: age=youth, income=medium, student=yes, credit_rating=fair
n = [[0, 1, 1, 0]] # 0=youth, 1=medium, 1=yes, 0=fair

# التنبؤ
p = m.predict(n)

# عرض النتائج
if p[0] == 1:
    print("Predicted Class: Yes")
else:
    print("Predicted Class: No")

```

Ensembles

1. What is Ensembles?

Ensembles are a category of machine learning methods that combine multiple machine learning models to create more powerful and robust predictive models.

Two ensemble methods have proven particularly effective across a wide range of datasets for both classification and regression tasks. Both of these methods use **decision trees**.

These two highly effective models are:

1. Random Forests.
2. Gradient Boosted Decision Trees.

1. Random Forests (الغابات العشوائية)

Random Forests are an ensemble method designed to solve the primary drawback of individual decision trees: their tendency to overfit the training data. A random forest is a collection of decision trees, where each tree is intentionally made slightly different from the others. The core idea is that while each individual tree may predict well, it will likely *overfit a different portion of the data*. By averaging the results of many such independent trees, the overall amount of overfitting is reduced while the collective predictive power is maintained.

To achieve this differentiation, two key randomization mechanisms are injected during the tree-building process:

1. Data Sampling (Bootstrap Sampling): Each tree is built on a bootstrap sample of the original data. This sample is created by randomly drawing *n_samples* data points *with replacement*. This ensures that each tree is trained on a slightly different dataset, with some original data points repeated and about one-third left out.
2. Feature Selection (max_features): At every single node split, the algorithm randomly selects a subset of features (controlled by the **max_features** parameter) and only searches for the best split within that subset. This is repeated independently for every node, guaranteeing that each split uses a different subset of features.

The Role of max_features

The max_features parameter is critical as it controls the degree of feature-based randomness:

- High max_features (e.g., set to n_features): The trees will be quite similar and will fit the data easily using the most distinctive features.
- Low max_features (e.g., set to 1): The trees will be quite different, and each tree may need to grow very deep to fit the data well.

To make a final prediction using a random forest:

- **Regression:** The predictions from all individual trees are averaged.
- **Classification:** A "soft voting" strategy is used, where the probabilities provided by each tree for all possible outcomes are combined to determine the final class.


```

from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

# 1. تحميل البيانات
X, y = load_breast_cancer(return_X_y=True)

# 2. تقسيم البيانات
X1, X2, y1, y2 = train_test_split(X, y, random_state=0)

# 3. إنشاء النموذج وتدريبه
f = RandomForestClassifier(n_estimators=100, random_state=0)
f.fit(X1, y1)

# 4. التقييم
t1=f.score(X1, y1)
t2=f.score(X2, y2)
print("Accuracy on training set: {:.3f}".format(t1))
print("Accuracy on test set: {:.3f}".format(t2))

```

`n_estimators= 100` is number of DT

2. Gradient Boosting Regression Trees (GBRTs)

GBRTs are another powerful ensemble method used for both **regression** and **classification** tasks. In contrast to Random Forests, Gradient Boosting works in a **serial manner**. Each new decision tree is built sequentially and is specifically designed to **correct the mistakes** (or residuals/errors) made by the combination of all previous trees in the ensemble.

Key characteristics of GBRTs include:

- **Weak Learners:** The models use many **simple models** (or weak learners), typically very **shallow trees** (often of depth one to five).
- **No Randomization:** By default, GBRTs rely on **strong pre-pruning** rather than randomization to achieve model simplicity.
- **Efficiency:** Using shallow trees makes the model smaller in terms of memory and allows for **faster predictions**.

Performance and Parameters

GBRTs are frequently the winning entries in machine learning competitions and are widely used in industry due to their high predictive power. They can often provide **better accuracy** than Random Forests if their parameters are tuned correctly, though they are generally **more sensitive** to these settings.

The most important parameters are:

- **n_estimators:** Increasing the number of trees adds more chances for the model to correct mistakes, thereby increasing **model complexity**.

- **learning_rate**: This controls how strongly each new tree attempts to correct the mistakes of the previous ensemble. A higher **learning_rate** leads to stronger corrections and a more complex final model.

نستخدم الامر

```
model = GradientBoostingClassifier(n_estimators=5, learning_rate=0.001, max_depth=2,
    random_state=0)
```

1. n_estimators= عدد الأشجار, # عدد الأشجار في النموذج
2. learning_rate= معدل التعلم, # قوة كل شجرة جديدة في تصحيح الأخطاء
3. max_depth= عمق الشجرة, # أقصى عمق لكل شجرة
4. random_state= رقم عشوائي, # لتثبيت النتائج عند كل تشغيل
يمكن عدم كتابته في البرنامج ليتم تعيينهم افتراضي كما في البرنامج التالي

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

# 1. تحميل البيانات
X, y = load_breast_cancer(return_X_y=True)

# 2. تقسيم البيانات
X1, X2, y1, y2 = train_test_split(X, y, random_state=0)

# 3. إنشاء النموذج وتدريبه
f = GradientBoostingClassifier(random_state=0)
f.fit(X1, y1)

# 4. التقييم
t1=f.score(X1, y1)
t2=f.score(X2, y2)
print("Accuracy on training set: {:.3f}".format(t1))
print("Accuracy on test set: {:.3f}".format(t2))
```